The delegation algorithm on one page

Setting. Two components run in series. The planner M_1 reads an input u (fence, home, goal) and writes a mission w (a short list of waypoints). The controller M_2 reads the mission w and the environment e (start position, tick lengths) and writes a target y_t at every tick $t = 0, \dots, H$. We are given a top-level contract (A, G), where A is a condition on (u, e) and G is a condition on (u, e, y), both PolySTL formulas. We want a planner contract (A_1, G_1) with A_1 a condition on u and G_1 a condition on (u, w), and a controller contract (A_2, G_2) with A_2 a condition on w and G_2 a condition on (w, e, y), such that

$$\text{(D1) } A \subseteq A_1, \qquad \text{(D2) } G_1 \subseteq A_2, \qquad \text{(D3) } A \cap G_1 \cap A_2 \cap G_2 \subseteq G.$$

These three conditions are all that is needed. If the planner meets (A_1, G_1) and the controller meets (A_2, G_2), then on any input allowed by A the planner produces a mission in G_1 (by D1), that mission is accepted by the controller (by D2), the controller's targets lie in G_2, and D3 turns all of this into G.

Idea. The controller already exists as code, so its contract is read off the code and certified once. The planner does not exist yet, so its contract is whatever is then forced by D1, D2 and D3, and that is a formula, computed without any code.

Step 1. Read the controller code. Write down three conditions. V is the set of missions the code accepts (its input guards, for example leg lengths between $\ell_{\min}$ and $\ell_{\max}$). P is the set of environments the code relies on (its clamps and parameter ranges, for example the tick length within $[dt_{\min}, dt_{\max}]$). T_ρ says that every target stays within distance ρ of the mission path and that some target comes within η of the last waypoint. Fix a list of candidate radii $\rho_1 < \rho_2 < \cdots < \rho_k$, starting at the largest deviation seen when the code is run on a few test missions.

Step 2. Certify the controller, once. For a candidate ρ, ask a bounded model checker whether some run of the controller code over H ticks, with the mission and the environment free, has $w \in V$ and $e \in P$ and leaves the tube T_ρ. Binary search the list for the smallest ρ^* with no such run. Set

$$A_2 := V, \qquad G_2 := \{(w, e, y) \ : \ e \notin P \ \text{ or } \ y \in T_{\rho^*}(w, e)\}.$$

This is the only step that touches code. It is done once per controller and horizon, not once per planner.

Step 3. Planner assumption. A_1 is the part of A that speaks about u alone, that is A with e projected out. D1 holds by construction.

Step 4. Planner guarantee. The mission must be such that the whole tube, under every environment the top-level contract allows, satisfies the top-level guarantee,

$$G_1(u, w) \ := \ w \in V \ \text{ and } \ \forall e \, \forall y \ \big[\, A(u, e) \wedge P(w, e) \wedge y \in T_{\rho^*}(w, e) \ \Rightarrow \ G(u, e, y) \,\big].$$

D2 holds because G_1 requires $w \in V = A_2$. D3 holds because the bracket is exactly the statement that A, A_2 and G_2 together imply G. G_1 is the largest set with these two properties, so nothing is given away relative to the controller contract of step 2. For a convex polygon fence and a tube the quantifiers disappear by hand. If G says "every target inside the fence and some target within ε of the goal" and A says "start within δ of home", then G_1 reads

$$w \in V, \qquad \text{every waypoint and the home point lie at least } \rho^* \text{ (resp. } \rho^* + \delta) \text{ inside the fence,} \qquad \|w_{\text{last}} - g\| \le \varepsilon - \eta,$$

plus the leg conditions of P applied to every start within δ of home. No solver is involved.

Step 5. Feasibility and discharge. Let *Bad* be the set of inputs $u \in A_1$ for which no mission w satisfies $G_1(u, w)$. If *Bad* is empty, a planner exists. Write the planner and verify it against (A_1, G_1) alone, which is a static formula over u and w with no ticks and no controller code. Every later planner revision, and every later query with the same controller and horizon, repeats only this cheap step.

Worked instance. Square fence $[-s, s]^2$, home h, goal g, start within 1 m of h, legs between 5 and 100 m, $\rho^* = 1$ m, $\eta = 0.5$ m, $\varepsilon = 3$ m. Then G_1 is the conjunction of $5 \le \|w_2 - w_1\|_\infty \le 100$, $6 \le \|w_1 - h\|_\infty \le 99$, $\|w_1\|_\infty \le s - 1$, $\|w_2\|_\infty \le s - 1$, $\|w_2 - g\| \le 2.5$. At $s = 50$ the set *Bad* is $\|h\|_\infty > 48$ or $\operatorname{dist}(g, [-49, 49]^2) > 2.5$; for every other input the mission $w_2 :=$ nearest point of $[-49, 49]^2$ to g, $w_1 :=$ midpoint of h and w_2 satisfies G_1.

What it costs and what it loses. Step 2 costs about one monolithic check of planner plus controller. Each planner check afterwards costs a fraction of that, so delegation pays from the second accepted planner or query sharing one controller certification. It loses the missions that the real code would fly safely but the tube cannot prove, which is at most about ρ^* of fence margin at the corners.